Recursion design record

Status: accepted; implementation in vertical slices. This record fixes the execution substrate and the versioning for `docs/recursion.typ`. It is the design the complete plan is built on. No feature is deferred for code size.

1. UnitGroup and Unit

Recursion is a property of a binding group. The executable records are:

```
Unit
    Name         : string
    Arity        : int
    Captures     : slot list
    Instructions : Instruction list
    SourceHash   : string

UnitGroup
    Name           : string
    Units          : Unit list
    InitOrder      : name list
    Classification : BindingClassification list
```

- A group is one strongly connected component of the definition graph. A self-recursive function is a group of one; `let rec ... and ...` is a group of several. The group is the recursion unit, not a special opcode.
- `Arity` and `Captures` are explicit. They are not inferred from instruction shapes. A unit call reads them and establishes the capture frame from the arguments and captured slots.
- `InitOrder` is the value-definition order. A function unit is created at its group's initialization; a value unit runs there. An out-of-order reference is a construction-time diagnostic.
- The unit body is the existing lowered instruction stream. There is no second execution engine.

2. Runtime

- The group resolver maps a name to a unit within the current group and to the enclosing groups. A self or mutual call is an ordinary call.
- A unit call pushes the argument/capture frame, runs the unit, and pops the frame. The existing `PushCapture`/`StoreSlot` prologue is reused.
- `MaxUnitDepth` bounds runaway recursion; exceeding it is a named failure.
- The callable contract (`docs/callable-contract.typ`) applies: a call resolves to a unit or a word; it never resolves to a bare lexical value.
- A group is initialized once per image load, in `InitOrder`; a failed initialization is a named diagnostic and the image is not executed.

3. Image and package encoding

- `PackageUnit` gains `Arity` and `Captures`; the package carries `UnitGroup` records. The existing `PackageUnit.Dependencies` remains the graph edge data.
- The kernel image unit gains `Arity` and `Captures`; the image carries the groups.
- This is an image-schema change. The image schema moves to **2** and the bundle version `SCALE_ABI_VERSION` moves to **2**. The header, the kernel export, the C adapter header, the conformance harness, the kernel AOT gate, and every adapter README move together. A version mismatch is a deployment error, never negotiated.
- The schema change is in one slice: header/version, conformance, image reader/writer, round-trip test, and the old-image refusal. The old image is refused by name, not guessed.

4. Book recursion

- A book is a module group. All definitions form one recursive group by default; the loader computes the graph and the `InitOrder`.
- The book surface has two markers: `logic` for an ordinary definition and `cycle` for a recursive definition. `and` groups `cycle` definitions F#-style: `cycle even n = odd(n). and odd n = even(n).` A `logic` body must not call its own name; a `cycle` body may call any name in its cycle group.
- The marker is checked against the computed group; a disagreement is a diagnostic. The marker never creates a second semantics.
- D188′s refusal is replaced by real recursive groups. A recursive call is never a silent closure result.

5. Our own F# abstraction

- Define a small, versioned F# subset abstraction for the Prelude and F# packages: modules, `let`/`let rec`/`let rec ... and ...` groups, functions with arity and lexical parameters, calls (including higher-order), literals, pattern matching, source ranges, and `BindingClassification`.
- A build-time classifier reads F#'s classification with FCS and emits the abstraction. A translator or a hand port lowers it to `UnitGroup`/`Unit`.
- FCS is a build-time dependency. The AOT runtime and the product do not link it; they consume the versioned classification and groups. The full read-off, manifest, tiers, and diagnostics are still implemented.
- The abstraction is not a second runtime. It lowers to the same groups and units as books and `define` words.

6. Diagnostics and versioning

- Scale diagnostics carry Scale source ranges. F# diagnostics (FS0040, unused-`rec`, value restriction) are mapped at build time through the diagnostic mapping table.
- `DeclaredReason` is an enumerated set with an `other` fallback that requires review.
- The classification record, the F# abstraction, the unit/group record, the image schema, and the bundle version are all versioned. A skew is a named refusal. The ABI/image version and the classification version are separate numbers with separate refusal paths.

7. Implementation slices

1. **Slice 1: schema and version.** `Unit`/`UnitGroup`, `Arity`/`Captures`, image/package encoding, image schema 2, `SCALE_ABI_VERSION` 2, header/adapter/ conformance updates, old-image refusal, round-trip tests.
2. **Slice 2: group runtime.** Group resolver, unit call frame, `InitOrder`, depth guard, named failures; tests.
3. **Slice 3: book recursion.** Automatic groups, book markers, marker comparison, diagnostics; replace D188′s refusal; tests.
4. **Slice 4: logic and cycle definitions.** Parser/lowering/word table, plain-name self-reference, self and mutual cycle groups; tests.
5. **Slice 5: our own F# abstraction.** Subset type, round-trip, build-time classifier/translator, manifest and tiers, diagnostic mapping, IPC metadata; tests.
6. **Slice 6: full acceptance and specification.** The complete acceptance list in `docs/recursion.typ`, the status matrix, counts, and the line counts.

Each slice follows docs → tests → implementation → specification. No slice reduces a later slice's contract.

8. Quality bar

- One substrate: books, `define`, the F# abstraction, and the Prelude port use the same groups and units.
- One callable contract: no parallel call path and no silent value push.
- FCS is build-time; the AOT runtime is complete without it.
- Explicit markers are optional refinements; automatic book recursion is the default; neither is a second semantics.
- Every failure is a named diagnostic with a source range; every version skew is refused.
- The implementation is sliced for review, not scope.

9. Slice 1 status

Slice 1 is implemented (D192). The image and package schema is 2; a unit carries `Arity` and `Captures`, and a `UnitGroup` carries `Units`, `InitOrder`, and a `BindingClassification` per unit. The image loader and package loader validate the group's unit set, order, and classification, and refuse a schema-1 document by name. `SCALE_ABI_VERSION` is 2, and the header, C adapter, conformance harness, READMEs, and image schema agree.

The group runtime, book recursion, and `define` markers are not part of this slice; D188′s refusal and the current VM remain until Slice 2/3.

10. Slice 2 status

Slice 2 is implemented for function groups (D193). `CallWord` resolves through the unit's group, pops the unit's arity, appends its declared captures from the caller's frame, and runs the unit; `CallClosure` uses the frame the closure already established and does not re-frame the target. `MaxCallDepth` bounds a runaway call, and `UnitArityMismatch`, `UnitCaptureMissing`, and `EntryUnitArity` are named failures. `InitOrder` is validated at load and used by the group resolver.

Value-unit initialization and book/`define` recursion are Slice 3 work. This slice deliberately does not execute value units or replace D188′s refusal.

11. Slice 3 status

Slice 3 is implemented (D194). A book is one recursive group; the loader compiles each definition to an image unit with explicit arity and no lexical captures, and calls it through the group runtime. Self, forward, and mutual calls work. The optional `logic` and `cycle` markers are checked against the definition's book calls; a `logic` self-call and a `cycle` with no book call are named refusals. D188′s refusal is retired, and a runaway recursion is bounded by `MaxCallDepth`.

`logic`/`cycle` definition words are Slice 4; the F# package classification and F# abstraction are Slice 5.

12. Slice 4 status

Slice 4 is implemented (D195). The core definition words are `logic` and `cycle`. `logic name { body }` stores a non-recursive word and refuses a body that calls its own name. `cycle name { body }` stores a recursive word; the body uses the plain name. `cycle first { body } and second { body }` stores a mutual group, and every group name is in scope in every body. There is no `recurse` word. The old stack form `body "name" define` remains.

The definition words are core-surface parsing words; the prose surface defines books with `logic`, `cycle`, and F#-style `and`. The F# package classification and F# abstraction are Slice 5.

13. Slice 5 status

The core F# abstraction and classification path are implemented (D196). `FSharpAbstraction` carries modules, bindings, parameters, expression bodies, calls, source ranges, and the F# recursion kind.

The manifest supports `automatic`, `mixed`, and `strict` modes; an override must agree or carry a `DeclaredReason`. The classifier emits `BindingClassification`, records accepted overrides, maps F# diagnostics, and emits `(name, kind, version)` metadata. The abstraction lowers to the same package unit/group substrate and round-trips through the package document.

The build-time FCS adapter is `tools/fsharp-classifier.fsx`. It emits the `ReadOffModule` JSON with bodies for the supported expression subset: literals, names, calls, lambdas, `let`, `if`, sequential blocks, and tuple/list blocks. `FSharpAbstraction.fromReadOff` combines the read-off with a hand-supplied body when one is present. An unsupported F# expression or pattern is a named refusal; it is never guessed. The product and AOT runtime do not link FCS.